



SISTEMAS DISTRIBUIDOS

PROYECTO FINAL



Mini Banco Concurrente Distribuido en la Nube

Elaborado por: M. en C. Ukranio Coronilla

El objetivo del proyecto es diseñar e implementar un sistema bancario distribuido basado en microservicios utilizando Java y servicios en la nube, capaz de soportar operaciones concurrentes masivas de manera confiable, consistente y escalable.

El sistema simulará un banco distribuido con múltiples clientes concurrentes realizando operaciones financieras sobre sus cuentas. Se les proporcionará un conjunto de miles de registros que incluyen un identificador, nombre completo y saldo para utilizarlo en las pruebas y que todos tengan la misma base de datos. En las pruebas sólo se soporta la operación de transferencia de dinero entre las cuentas, de modo que al terminar una prueba el monto total de las cuentas debe permanecer constante. Por otra parte, todos los usuarios requieren autenticación con JWT para tener acceso a los endpoints seguros.

Se adjunta el programa generador de registros **GeneradorRegistros.java** y los archivos **nombres.txt** y **apellidos.txt** para ejecutarse de manera similar a como se hizo en la práctica “Estrategia de partición de datos”.

Para minimizar los costos y como un reto para evaluación de conocimientos y competencias, se tiene prohibido utilizar el servicio Cloud SQL o cualquier otro servicio similar en otras plataformas para almacenar la base de datos, así como cualquier SGBD embebido. En su lugar y para el almacenamiento de información y ejecución de servicios de todo el proyecto podremos utilizar única y exclusivamente dos instancias e2-standard-2 y una instancia e2-standard-4 de la serie E2.

e2-standard-2

2 CPU virtuales (1 núcleos), 8 GB de memoria

e2-standard-4

4 CPU virtuales (2 núcleos), 16 GB de memoria

Importante: No puede usarse otro tipo de instancia, en el caso de utilizar otra plataforma de nube será necesario utilizar un equivalente, pero con un conjunto de pruebas benchmark que lo demuestren.

Por sencillez se recomienda configurar en las instancias:

nodo-1 ← líder fijo instalado en la instancia e2-standard-4

nodo-2 ← réplica

nodo-3 ← réplica

Las solicitudes de lectura y escritura siempre llegarán a la instancia que contiene el nodo 1, la cual es una instancia que supondremos casi infalible para este proyecto y cuya IP será compartida para quien desee acceder al mini banco.

Para probar la tolerancia a fallos, apagaremos una réplica y debe funcionar el sistema, apagamos la otra réplica y debe seguir funcionando. Encendemos una de las réplicas y deberá actualizar su base de datos para mantener la consistencia con el líder y finalmente encenderemos la otra réplica con lo cual deberemos tener el sistema consistente en los tres nodos.

Debe existir un log de todas las transacciones hasta el momento con su número de secuencia, las cuales estarán almacenadas en Cloud Storage y sirven para recuperar el estado completo de la base de datos en el caso que fallen todos los nodos en algún momento. Se recomienda el uso de Pub/Sub pero cada equipo puede diseñar su solución como desee.

Cuando una réplica vuelve a la vida, no puede simplemente borrar todo y pedir la base de datos completa del Cloud Storage. Debería manejar un número de secuencia de transacción. Al conectarse, la réplica dice: "Me quedé en la transacción 4500", y debería recibir de la 4501 en adelante.

Concurso

Cada equipo tendrá que elaborar un programa generador de carga en Java que deberá lanzar una carga masiva durante 1 minuto consistente en operaciones de las cuales un 80% son lecturas de saldo y un 20% son operaciones de transferencias entre registros. Dicho programa se ejecutará en una instancia en la nube del tipo y serie que se desee.

Dado que todos los equipos deben tener la misma base de datos inicial, después de 1 minuto de carga masiva la base de datos debe terminar en un estado consistente conocido por el programa generador de carga. Si el programa generador de carga detecta una inconsistencia deberá poder indicar el registro donde está la inconsistencia.

En caso de que no detecte inconsistencias deberá imprimir el número de transacciones exitosas y el número de lecturas exitosas conseguidas en el minuto de prueba.

Se obtendrán los tres mejores sistemas considerando a los que hayan obtenido el mayor número de transacciones exitosas, así como el mayor número de lecturas exitosas durante la carga de trabajo, teniendo cuatro veces más valor las transacciones que las lecturas. Además, se sumarán las transacciones exitosas para los tres casos de prueba:

1. Nodos 1, 2 y 3 activos.
2. Nodos 1 y 2 activos.
3. Nodo 1 activo.

A los miembros del equipo ganador se les otorgarán dos puntos en la **calificación final del curso**, un punto en la **calificación final del curso** al equipo que quede en segundo lugar y medio punto en la **calificación final del curso** al equipo que quede en tercer lugar.

Definición Estándar de la API

Para que el sistema de un equipo A pueda ser sometido a pruebas por el programa de pruebas del equipo B, todos deben implementar exactamente los mismos endpoints.

Endpoints de Autenticación y Registro

Registro de usuarios como lo vimos en clase:

```
POST /api/register
```

Retorna el token JWT:

```
POST /api/login
```

Endpoints Transaccionales (Requieren JWT en el Header `Authorization: Bearer <token>`)

Endpoint que retorna el saldo actual de un usuario con identificador id:

```
GET /api/accounts/{id}
```

El cual devuelve en formato JSON:

```
{
  "id": 125,
  "propietario": "JUAN MOLINAR HERNANDEZ",
  "balance": 15750.25
}
```

Endpoint para transferencia entre dos cuentas:

```
POST /api/transactions/transfer
```

En el cuerpo de datos debe enviar en formato JSON un mensaje como el siguiente:

```
{"sourceAccountId": "123", "targetAccountId": "456", "amount": 200.00}
```

Es muy importante que el JSON de respuesta o el código de estado HTTP (ej. 200 OK, 400 Bad Request, 401 Unauthorized) coincida con el estándar REST.

Dashboard de Observabilidad

Debe existir un dashboard de una sola página web que permita observar el estado completo del sistema distribuido. La información actualizada puede observarse a partir de dar click en un botón “Refrescar”, o por 1.5 puntos extra en el proyecto si implementan un flujo reactivo/en tiempo real.

En la página se debe mostrar **para cada uno de los nodos** el estado del nodo, número total de cuentas, saldo total, numero de transferencias realizadas, , identificador de la última transacción realizada, porcentaje de uso de CPU, porcentaje de uso de RAM y porcentaje de uso de Disco Duro.

Del Cloud Storage se debe obtener el número de transacciones que van hasta el momento almacenadas.

Esta información va a permitir a los programadores el desarrollo adecuado del proyecto y también resolver dudas cuando otros equipos prueben su sistema. Es también posible agregar información adicional al dashboard si así se desea.

Evaluación

Este proyecto será programado en equipos de tres alumnos que serán asignados de entre mis tres grupos de Sistemas Distribuidos y cuya lista será publicada en teams. Por su complejidad equivale a dos proyectos individuales, en este caso la calificación obtenida se asigna al proyecto 6 y 7.

Todo el código backend del proyecto debe estar en lenguaje Java (no es válido el uso de Python u otro lenguaje de programación salvo lo necesario en el dashboard).

Los detalles adicionales que no se hayan considerado se publicarán en la plataforma de teams el día de la evaluación.

Importante: Todos los puntos del proyecto serán revisados detalladamente con un checklist, por lo que para obtener 10 se requiere cumplir con todos ellos.

La revisión del proyecto final (explicación del código y funcionamiento) se realizará únicamente el jueves 25 de junio comenzando a las 8:00 AM y hasta que termine la evaluación de todos los equipos.

Cada equipo deberá entregar todo el código del proyecto final con la consideración de que, si se detecta un plagio, ambos equipos (plagiado y plagiario) tendrán una calificación de cero.

Si un alumno no trabajó o hubo quien trabajó menos que los demás tendrán que hacérmelo saber en el momento de la revisión para ponderar la calificación de los miembros del equipo.